

Lab\_4\_VibeCoding\_handout.pdf

PDF

Pasted text.txt

Document

fix the bug in the terminal first

The terminal bug is clear. Your code has **not reached the game yet**—`pip install -r requirements.txt` is failing while installing `pygame`.

Your terminal shows **Python 3.14.5**, and the required `pygame` version is being downloaded as a source archive (`pygame-2.6.1.tar.gz`). Its build process then fails because it tries to import the removed `distutils.msvccompiler` module.

## Fix it first

Since this is a Windows project and you need to get the assigned Python game running, the cleanest fix is to use **Python 3.13** for this project rather than Python 3.14.

### 1. Check whether Python 3.13 is already installed

Run this in your current terminal:

```
py -0p
```

If you see something like:

```
-3.14
```

```
-3.13
```

then run:

```
py -3.13 -m venv .venv
```

Then activate it:

```
.venv\Scripts\Activate.ps1
```

You should see:

```
(.venv) PS C:\Users\Gitanjali\Desktop\42_marble-tilt-maze>
```

### 2. Upgrade pip inside the new environment

```
python -m pip install --upgrade pip
```

### 3. Install the requirements again

```
python -m pip install -r requirements.txt
```

### 4. Verify pygame

```
python -c "import pygame; print(pygame.version.ver)"
```

If successful, you should get a pygame version instead of an error.

### 5. Run the game

Before changing any code, the lab specifically asks you to run the Python game and record the **10-second before video**.

So after pygame installs successfully, run the project's normal command. If the README says, for example:

```
python main.py
```

use that command.

---

### Important: don't try to fix the **distutils** error manually

The important part of your error is:

```
ModuleNotFoundError: No module named 'distutils.msvccompiler'
```

and then:

```
ModuleNotFoundError: No module named 'setuptools._distutils.msvccompiler'
```

You already upgraded pip, but that did **not** solve it; your second installation attempt produced the same failure.

**Do not modify the game code yet.** First get the environment working.

Run:

```
py -0p
```

and paste the output here. I'll give you the **exact next terminal command** based on what Python versions you have installed.

## Task 3 — Add Replay + Difficulty Selection

The README defines Task 3 as allowing the player to replay after the end screen by choosing **Easy, Medium, or Hard**, with different **tilt strength, friction, and time limit**, or exit.

Use this as your next Vibe Coding prompt:

### Task 3 Prompt

Implement Task 3 only.

Add a replay system after the existing game-over screen.

Requirements:

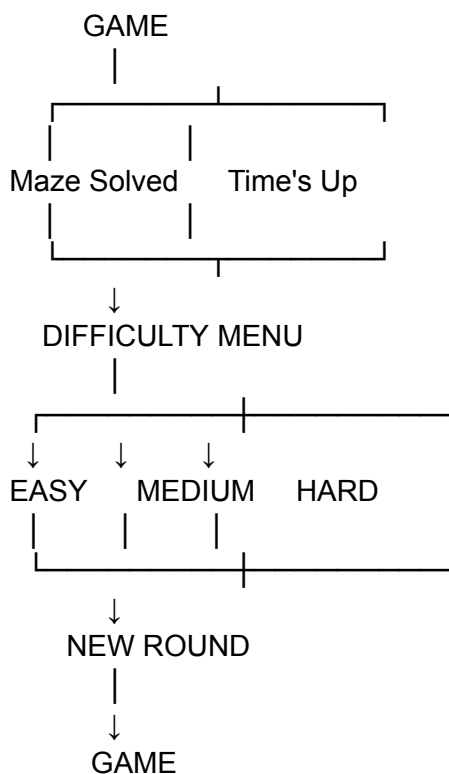
1. After the game-over screen, display a difficulty selection menu with:
  - Easy
  - Medium
  - Hard
  - Exit
2. Each difficulty must change:
  - tilt strength
  - friction
  - time limit
3. Use sensible difficulty values while keeping the existing gameplay mechanics unchanged.
4. When the player selects Easy, Medium, or Hard:
  - Reset the marble to its starting position.
  - Reset its velocity to zero.
  - Reset the timer.
  - Reset `game_over`.
  - Reset `result`.
  - Reset `finish_time_ms`.
  - Start a completely new round using the selected difficulty.
5. The maze and goal should remain unchanged between rounds.
6. The game-over screen from Task 2 should lead naturally into the difficulty-selection screen.
7. Keyboard controls should be simple:
  - E = Easy
  - M = Medium
  - H = Hard
  - ESC = Exit
  - You may also support arrow keys/number keys if useful, but keep the controls clear on screen.
8. Do not implement Task 4 sound effects yet.
9. Preserve the Task 1 collision fix and Task 2 game-over functionality.

10. Avoid duplicating the entire game loop. Keep the existing `main.py` game loop structure if possible.
11. Organize the difficulty configuration cleanly, preferably using a dictionary or similar structure.
12. Explain the architecture and changes before providing the code.

After implementing it, make sure the player can complete multiple rounds without restarting the Python program.

## Expected flow

Your game should now behave like:



## One important check

Make sure the AI **resets the timer correctly**.

A common mistake is resetting:

```
self.game_over = False
```

but forgetting:

```
self.start_ticks = pygame.time.get_ticks()
```

If `start_ticks` isn't reset, the new round can immediately inherit the old round's elapsed time.

Also make sure the marble gets reset:

```
self.marble.x = 50
self.marble.y = 50
self.marble.vx = 0
self.marble.vy = 0
```

After the AI gives you the Task 3 implementation, test:

1. Finish a game.
2. Select **Easy**.
3. Play another round.
4. Finish it.
5. Select **Hard**.
6. Verify the new round starts correctly.
7. Exit with **ESC**.

Then we can move to **Task 4 — sound feedback**.

move on to task 4

## Task 4 — Add Sound Feedback

This is the final task in the README. It requires basic sound effects for **wall bounce, reaching the goal, and timer expiration**.

Use this prompt in your Vibe Coding tool:

Task 4 Prompt

Implement Task 4 only.

Add basic sound feedback to the Marble Tilt Maze.

Required sound effects:

1. A bounce sound when the marble collides with a wall.
2. A success sound when the marble reaches the goal.
3. A timeout sound when the timer reaches zero.

Requirements:

- Use Pygame's audio system (`pygame.mixer` / `pygame.mixer.Sound`).
- Initialize the mixer safely so the game does not crash if audio is unavailable.

- Keep the existing Task 1 collision detection unchanged.
- Keep the existing Task 2 game-over screen unchanged.
- Keep the existing Task 3 replay and difficulty system unchanged.
- Play the bounce sound only when an actual wall collision occurs, not every frame while the marble is touching a wall.
- Play the goal sound exactly once when the marble reaches the goal.
- Play the timeout sound exactly once when the timer expires.
- Prevent sounds from repeatedly triggering during the same event.
- Store sound objects as attributes of `GameEngine`.
- Use a clean fallback if sound files are missing, rather than crashing the game.
- Do not change the existing gameplay physics.
- Do not duplicate the game loop.
- Keep the existing project structure as much as possible.

If sound files are not already present, explain where they should be placed and what filenames should be used.

After implementing the feature, explain exactly which files were changed and how each sound is triggered.

## Recommended sound structure

If the AI asks you to add sound files, use something like:

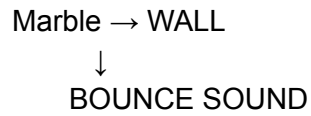
```
42_marble-tilt-maze/
|
|-- main.py
|-- requirements.txt
|-- README.md
|
|-- game/
|   |-- game_engine.py
|   |-- marble.py
|   |-- wall.py
|
|-- assets/
|   |-- sounds/
|       |-- bounce.wav
|       |-- goal.wav
|       |-- timeout.wav
```

## Test all three events

After implementing Task 4:

### 1. Wall collision

Move the marble into a wall:



Make sure the sound doesn't continuously play while the marble is pressed against the wall.

### 2. Goal

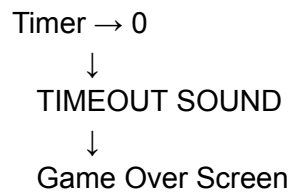
Reach the green goal:



The goal sound should play **once**.

### 3. Timeout

Allow the timer to reach zero:



The timeout sound should also play **once**.

## Final testing checklist

Once Task 4 works, test the complete flow:

| Feature   | Test                                      |
|-----------|-------------------------------------------|
| Collision | Marble correctly bounces at walls/corners |
| Timer     | Countdown works                           |
| Goal      | Goal ends the round                       |

|               |                          |
|---------------|--------------------------|
| Game Over     | Result appears on screen |
| Replay        | New round starts         |
| Easy          | Difficulty changes       |
| Medium        | Difficulty changes       |
| Hard          | Difficulty changes       |
| Exit          | ESC exits                |
| Bounce sound  | Plays on collision       |
| Goal sound    | Plays once               |
| Timeout sound | Plays once               |

The assignment then requires the **after video** showing the completed game, and the submission includes the before/after videos, updated code, and chat history.

After Task 4, **don't add extra features unless your README requires them**. Your four assigned tasks are now complete.

how to convert the chat history to docx

For your lab, the README says the submission needs the **complete chat history** as part of the deliverables.

## Easiest way to get a DOCX

If you're using ChatGPT in the browser:

1. Open this conversation.
2. Use the **conversation menu** (...) near the chat title.
3. Look for **Export / Share** options.
4. If the available export gives you HTML, Markdown, or another format, you can convert that file to **.docx**.

## I can create the DOCX for you

If you want the **chat history from this conversation formatted as a proper DOCX**, I can create it with sections such as:

LAB 4 – VIBE CODING  
Marble Tilt Maze

1. Initial Terminal Error  
User prompt  
ChatGPT response
2. Terminal Fix  
User prompt  
ChatGPT response
3. Task 1 – Collision Detection  
User prompt  
ChatGPT response
4. Task 2 – Game Over Screen  
User prompt  
ChatGPT response
5. Task 3 – Replay & Difficulty  
User prompt  
ChatGPT response
6. Task 4 – Sound Feedback  
User prompt  
ChatGPT response
7. Final Result

I can also include the **prompts you used and the corresponding generated instructions**, which is useful because the lab specifically asks you to demonstrate the LLM-assisted iterative process.

If you want **the complete chat history in DOCX**, say “**create the docx**” and I'll generate the file.